

Session 7 · Deck 7.B1

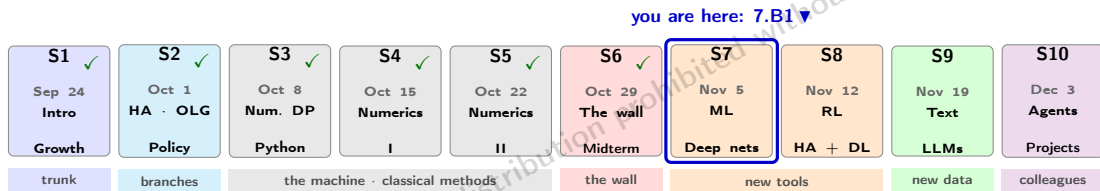
The Growth Model via Euler Residuals

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 7.B1 the model as a loss function · 7.B2 deep VFI and actor-critic · 7.B3 structure and validation.

Block A Built the Instrument. Block B Points It at a Model.

Everything in Block A was about *functions*: what a network can represent, how its parameters are found, what an architecture assumes. Not one frame mentioned an economy.

Block B asks the question that makes any of it useful here:

What do you train on when there is no data?

The answer is the one thing a macroeconomist always has and a statistician never does: **the model's own equilibrium conditions**. They supply the target. Every state is a free observation, and there are as many as you care to draw.

The three decks

7.B1 — write the first-order conditions as a loss, and minimise it. *One network, and the FOCs must be derived.* **7.B2** — write the Bellman equation as a loss instead. *Two networks, and no FOCs needed.* **7.B3** — make the network obey the theory by construction, and then check whether any of it worked.

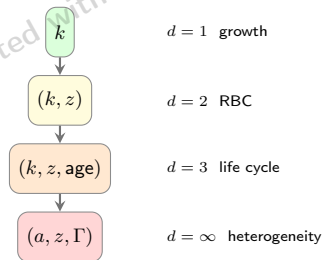
The Workhorse, and Why It Is One

The model is the one from 1.B1, and it has not changed:

$$V(k) = \max_{k'} \{u(f(k) + (1 - \delta)k - k') + \beta V(k')\}.$$

It is a workhorse because every research direction in macroeconomics is a modification of it: add a shock and it is the RBC model; add age and it is a life-cycle model; add idiosyncratic risk and incomplete markets and it is Aiyagari.

And each of those directions adds state variables.



The N^d Explosion

A grid places N points *per dimension*, so d dimensions cost N^d . With $N = 100$:

d	points	
1	10^2	trivial
2	10^4	manageable
5	10^{10}	infeasible
10	10^{20}	absurd

Linear in resolution, exponential in dimension.

Every extra state variable multiplies the cost by another N .

Ledger param_count.tensor_grid_points. Code: [s07_generate.py](#), run_param_count.

And it is worse than it looks

Even at a coarse three points per axis, $d = 20$ needs $3^{20} \approx 3.5 \times 10^9$ points; at five points, 9.5×10^{13} . **Sparse grids delay this and do not remove it** — 5.B3 measured exactly how much they buy.

Note what is *not* the problem: storage of the answer. The policy function is a smooth object. It is the **construction** that costs N^d , which is 7.A1's “the bottle-neck moves”.

What the Classical Toolkit Gives Up

Sessions 3–5 built four methods, and each bought its tractability with a concession:

- **grid VFI** — exhaustive, and N^d ;
- **perturbation** — gives up global validity for speed and dimension. 5.A6 measured the cost: 1.8 decades of accuracy at half the steady state;
- **Chebyshev** — superb on smooth functions, and 5.B2 showed what a kink does to it;
- **finite elements** — handles the kink, if you already know where it is.

The common structure

All four **choose the approximating family before looking at the solution**, and all four pay for dimension in the size of that family. 7.A2's Theorem 6 says this is not an accident of the four we picked: *any* basis fixed in advance is capped at $m^{-2/d}$.

So the opportunity is precise: replace the fixed family with a **learned** one, and replace the grid with **sampling**. That is the entire proposal, and the rest of this deck is its execution.

What Deep Learning Actually Offers Here

1. **Approximation that scales.** A network's parameter count grows with *width and depth*, not with N^d .

Adding a state variable adds one input — a row of weights, not a factor of N .

2. **Differentiation for free.** 4.A4's reverse mode gives the whole gradient for about the cost of one evaluation, so a loss built from equilibrium conditions can be differentiated without anyone writing a Jacobian.

3. **Hardware.** The arithmetic is dense matrix multiplication — what a GPU exists to do.

The AlphaGo reframing

Go has around 10^{170} states — a state space no grid will ever cover. What worked was not a better grid but a different posture: **sample states, evaluate them with a network, and improve the network from what the samples say.** A macro model is the same shape of problem, with a known transition and a known reward.

And one honest difference: we *know* our model — which is why this deck needs no exploration at all. S8 is what happens when you give that up.

The Running Example

The model solved in this deck is the one 3.A4, 4.B5, 5.A6 and 5.B6 already solved — **deliberately**, because it is the only way the comparison means anything.

Deterministic form, for the theory: $u = \log c$, $\delta = 1$, $f(k) = k^\alpha$. Then the policy is known in closed form,

$$k' = \alpha\beta k^\alpha,$$

which is 1.B1's benchmark: *if a method cannot recover this, do not trust it on anything harder.*

Quarterly stochastic form, for the solve: $\alpha = 0.33$, $\beta = 0.99$, $\delta = 0.025$, $\log z$ an AR(1) with $\rho = 0.95$, $\sigma = 0.007$, discretised by Rouwenhorst to seven states.

Calibration continues Session 4's, which is HW1 Q3's.

Two grounds truths

The closed form checks **correctness**. The classical solutions of 4.B5 and 5.B6 check **accuracy**, because they are themselves measured. A method that matches the closed form and then disagrees with a validated classical solve has a bug, not a disagreement.

$d = 1$ deterministic, $d = 2$ with the shock. **The easiest possible problem for a classical method** — which is the point of running it.

Two Roads from the Same Model

Bellman. Train networks to satisfy

$$V(k) = \max_{k'} \{u(c) + \beta V(k')\}.$$

Two networks — a value and a policy — and **no first-order conditions need to be derived**. Handles kinks and discrete choices. This is 7.B2, and it has a reinforcement-learning flavour.

Euler. Train one network to satisfy the first-order condition

$$u'(c) = \beta \mathbb{E}[u'(c')R'].$$

One network, and the FOCs must exist and be derived. Very precise where they do. **This deck.**

Which is which, in one word

Euler-residual training is **supervised**: regress a residual against zero. Actor-critic is **reinforcement**: a policy gradient on the Bellman objective. Both are, in the end, a PyTorch optimisation — and the loop is 3.B3's five lines.

Neither is right in general. The Euler road is better when the FOCs are clean; the Bellman road is better when they are not — which is most of the interesting models.

The Euler Equation, and Its Residual

With $k' = g(k)$ and $c = f(k) + (1 - \delta)k - g(k)$, the equilibrium satisfies

$$u'(c) = \beta u'(c') [\alpha g(k)^{\alpha-1} + 1 - \delta]. \quad (\text{EE})$$

Marginal cost of saving today equals discounted marginal benefit tomorrow.

Define the **Euler residual** of a candidate policy g :

$$\mathcal{R}(k; g) = u'(c) - \beta u'(c') [\alpha g(k)^{\alpha-1} + 1 - \delta].$$

The goal is to make $\mathcal{R}$ vanish at every economically relevant k .

You have met this object

5.B1 defined exactly this and called it the residual. The projection framework was: pick a family, then make $\mathcal{R}$ small. **Only the family is changing.** The residual, and what “solved” means, are unaltered.

In practice the residual is made **unit-free** — divided by $u'(c)$ — so that its size reads as a consumption error, which is 3.A3's convention and the one every number in this course uses.

Necessary, Not Sufficient

The Euler equation is **necessary**, not sufficient — 1.B1 said so: many paths satisfy it and most explode. A complete characterisation also needs the **transversality condition** $\lim_t \beta^t u'(c_t) k_{t+1} = 0$ and **feasibility**, $c \geq 0$, $k' \geq 0$.

Where a constraint binds, Kuhn–Tucker gives **three** conditions at once — $a \geq 0$, $b \geq 0$, $ab = 0$. The **Fischer–Burmeister** function

$$\varphi(a, b) = a + b - \sqrt{a^2 + b^2}$$

is zero *exactly* when all three hold, so **one smooth equation replaces the complementarity**. It drops straight into the residual, and autograd differentiates it like anything else.

Fischer & Burmeister (1992). The function is non-smooth only at the origin, which is where both conditions bind exactly.

Why this matters here

A network minimising $\mathcal{R}^2$ will happily find an explosive path if that makes the residual small: **the transversality condition is not in the loss**, and nothing in training imposes it. Here the bounded parameterisation does the work — the policy is confined to a compact capital interval, so an explosive path is not representable.

Here a is the multiplier on $k' \geq 0$ and b the slack. 2.A1 met φ as a **penalty**, $\lambda \sum \varphi(\cdot)^2$ — “how a neural solver enforces what it cannot impose”. This is the other half of that sentence.

From Residual to Loss

Three steps and the method is complete. **1.** Approximate the policy by a network, $g(k) \approx \mathcal{N}(k; \theta)$. **2.** Sample states $k_i \sim D(k)$ — uniform on a band, or ergodic. **3.** Minimise the mean squared residual

$$\mathcal{L}(\theta) = \mathbb{E}_{k \sim D} [\mathcal{R}(k; \mathcal{N}_\theta)^2] \approx \frac{1}{N} \sum_{i=1}^N \mathcal{R}(k_i; \mathcal{N}_\theta)^2,$$

with every derivative taken by autograd.

It is supervised learning

The “label” on every sampled state is **zero**. So 7.A3 applies unchanged — mini-batches, Adam, a schedule — with one difference worth savouring: **the labels are not noisy, and the training set is unlimited.**

The intuition is a mean-value one: a policy satisfying the Euler equation *on average* over the region that matters is close to the equilibrium policy, and a large average residual means optimality is violated somewhere.

This Is 5.B5, with One Substitution

Write the loss as the integral it is:

$$\mathcal{L}(\theta) = \mathbb{E}_{k \sim D} [\mathcal{R}(k)^2] = \int \mathcal{R}(k)^2 D(k) \, dk,$$

and compare 5.B5's weighted residual method, which chose θ to minimise

$$\int w(k) \mathcal{R}(k)^2 \, dk.$$

They are the **same object**, with $w(k) = D(k)$: **the sampling distribution is the weight function.**

The promise 5.B1 made

“Replace $\sum \theta_i \psi_i$ by $\mathcal{N}(x; \theta)$ and the framework is unchanged. That is S7.”

Here it is, discharged: **a neural Euler solver is least-squares projection on a learned basis, with the weight function chosen by where you sample.**

Read it both ways: D uniform on a grid $\approx$ **collocation** with equal weights; D the stationary distribution buys accuracy **where the model spends its time.**

Fernández-Villaverde & Guerrón-Quintana, *Solution Methods IV*. This is the frame 5.B1 promised.

What Is Genuinely Different from Collocation

Collocation and Galerkin require two choices made in advance: a **basis** $\{\psi_i\}$, and a set of **collocation points** or weight functions. The residual is then driven to zero at those points, or made orthogonal to those functions.

The network approach makes neither choice. There is no basis to select, and no node set: the residual is averaged over **sampled** states, so the weighting is implicit in the sampler.

	projection	network
family	chosen	learned
where	nodes	samples
weight	$w(\cdot)$, chosen	$D(\cdot)$, sampled
solve	root-find on θ	descent on θ
scaling	$m^{-2/d}$	see 7.A2

The last row is the whole bet, and 7.A2 was careful about it: the escape from $m^{-2/d}$ is real **and** conditional on the function being one a few ridge directions can express.

The Stochastic Case: an Expectation Inside the Residual

With a productivity shock the Euler equation carries a conditional expectation:

$$u'(c_t) = \beta \mathbb{E}_{z'|z} [u'(c_{t+1}) (\alpha k_{t+1}^{\alpha-1} z' + 1 - \delta)].$$

There is no closed form for it, so it has to be computed — inside the loss, at every step.

The blunt option is Monte Carlo over J draws:

$$\widehat{\mathcal{R}}(k_i) = u'(c_i) - \frac{\beta}{J} \sum_{j=1}^J u'(c'_{ij}) R'_{ij}.$$

Two layers of sampling

An **outer** average over states $k_i \sim D$ — that is the loss — and an **inner** average over shocks z' — that is the expectation. Different things, different remedies. Outer noise makes descent stochastic and is *useful*; **inner noise contaminates the residual itself**.

Monte Carlo error falls like $1/\sqrt{J}$, and that noise enters every gradient. 4.B2 had a better instrument.

Gauss–Hermite, and Why the Loss Stays Differentiable

1. **The problem.** A Monte-Carlo estimate puts noise *inside* the residual. Unlike 7.A3's outer sampling noise, which averages out and even helps, **inner noise corrupts the very quantity being driven to zero.**

2. **The fix.** 4.B2's quadrature: J **deterministic** nodes and weights,

$$\mathbb{E}_{z'|z}[h(z')] \approx \sum_{j=1}^J w_j h(z'_j),$$

exact whenever h is a polynomial of degree $\leq 2J - 1$ against the Gaussian weight. Smooth integrands are nearly polynomial, so **five to seven nodes match thousands of draws.**

For a log-normal shock, change variables to a standard normal and read the nodes off the table.

3. Why it is safe for autograd

The nodes and weights are **constants**, so the loss is a fixed weighted sum of smooth network outputs and the tape differentiates straight through. Mini-batching changes only *which* states enter the sum. And we sample **exogenous** shocks, never policy-dependent randomness — so no reparameterisation trick is needed. 4. **And in this deck.** The shock is a **seven-state Rouwenhorst chain**, so the expectation is an *exact finite sum* and none of this arises. **Gauss–Hermite is what you reach for when the shock is continuous.**

The Policy Network, and Feasibility by Construction

The architecture is unremarkable — a small MLP with a sigmoid on the output, σ_1 a ReLU or tanh:

$$\mathcal{N}(k; \theta) = \sigma_2(W_2 \sigma_1(W_1 k + b_1) + b_2).$$

What matters is what is done with it. The sigmoid gives $s \in (0, 1)$, so set

$$g(k) = s \cdot [f(k) + (1 - \delta)k - \varepsilon],$$

so that $0 \leq g(k) \leq f(k) + (1 - \delta)k$ **identically**.

Why this is not a detail

$u'(c) = c^{-\sigma}$ **diverges** as $c \rightarrow 0$: one infeasible sample gives an enormous residual, an enormous gradient, and a destroyed network. The bound is in the **parameterisation**, not bolted on afterwards, so an infeasible policy is not representable. The ε keeps marginal utility finite at the edge.

Alternatives: softplus (positive, smooth, unbounded above); hard clamp (simple, and *kills the gradient* at the boundary); log-parameterisation.

This is 7.B3's theme in miniature: a property imposed by architecture rather than hoped for.

The Training Loop

```
for it in range(iters):
    opt.zero_grad()                # 1 ZERO
    k = sample_states(batch)       # 2 FORWARD
    c = cash(k) * net(k)           # feasible
    kp = cash(k) - c
    rhs = 0.0
    for j in range(n_z):           # exact sum
        cp = cash(kp, j) * net(kp, j)
        rhs += Pi[:, j] * R(kp, j) / cp
    resid = 1.0 - c * beta * rhs
    loss = (resid**2).mean()        # 3 SCORE
    loss.backward()                # 4 BACKWARD
    opt.step()                     # 5 STEP
```

Recognise it

The five numbered comments are 3.B3's loop, **zero**, **forward**, **score**, **backward**, **step**, in that order and unchanged. **Everything else** — the four lines that build `resid` — is the Euler equation.

The network is called **twice** — at k and at k' — so the graph holds the policy composed with itself and the gradient flows through both. This is not a regression on a fixed target. **Pros:** one network, precise, grounded. **Cons:** no value function, and the FOCs must exist.

Where to Sample: the Choice That Is Easy to Miss

$D(k)$ is a **modelling decision**, not a technicality, because it is the weight function.

Uniform over a band. Simple, and it spends equal effort everywhere — including regions the economy visits with probability near zero. Accuracy is spread thin.

The ergodic distribution. Simulate the model under the current policy and train where it actually goes. Accuracy concentrates where it matters, and the objective is the economically meaningful one.

And the trap

The ergodic distribution depends on the policy, which is what is being learned. Train only where the *current* policy goes and it can settle into a small region, satisfy the Euler equation beautifully there, and be **arbitrarily wrong outside it** — with a small loss reporting success.

The practical answer is a **mixture** — mostly ergodic, part from a wider band. And always **evaluate somewhere other than where you trained** (7.B3).

The worked example below samples uniformly on $[0.5k^*, 1.5k^*]$, which is 5.B6's band exactly.

The Same Model, a Fifth Way

Everything above, applied to the quarterly RBC model of 5.B6 — the **same** Proj object, chain, band $[0.5k^*, 1.5k^*]$ and unit-free metric. Nothing is re-specified.

- **Network:** 3 hidden layers of 64, tanh; $c = \text{sigmoid}(\text{net}) * \text{cash}$, 8,961 parameters.
- **States:** 1,024 fresh uniform draws per step; **nothing is reused**.
- **Expectation:** exact over the seven states.
- **Optimizer:** Adam, lr $3e-3$ cosine-decayed to $1e-5$, 30,000 steps.

What is being tested

The **easiest** problem in the course for a classical method — one endogenous state, a smooth policy, a known band — and therefore the hardest case for the network. Running it here rather than on something flattering is the point.

Trained to a plateau, not a budget. The last tenth of training improved the loss by only 34%; a five-fold rise in parameters and iterations moved the headline number by 0.02 of a decade.

Ledger worked_nn_growth. The generator imports tools/figures/s05_generate.py, so 7.B1 and 5.B6 solve the same model by construction. Run on a compute node, CPU, float64. Code: [s07_generate.py](#), run_worked_nn_growth; L07 Part 5.

Did It Converge?

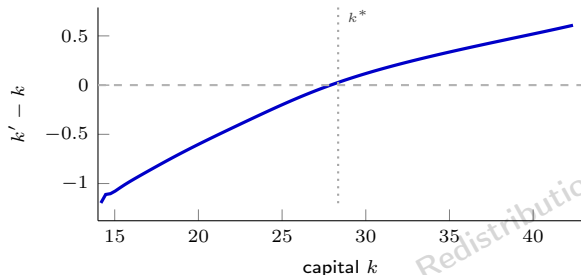


The loss falls by about four decades and then **flattens, noisily**. The noise is the outer sampling — a fresh batch of states every step — and it never disappears, because the loss is an expectation being estimated, not a fixed number being minimised.

This curve cannot tell you the answer is right. It is the training loss, measured on the states the optimizer just used. 7.A3's warning applies with full force, and the only honest verdict comes from evaluating somewhere else.

“One decade” is a factor of ten in the error (3.A1). Ledger `fig_nn_loss`, recorded every 1,000 iterations. Code: [s07_generate.py](#).

The Policy It Learned



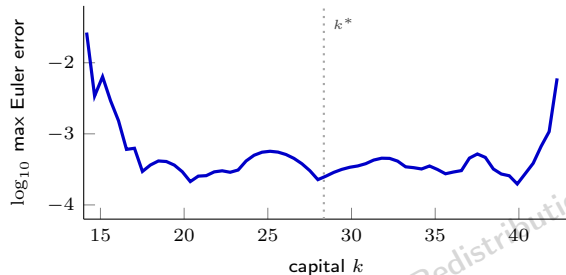
Plotted as **net saving** $k' - k$ at the middle shock state, because the level k' against k is visually a 45° line for every method and shows nothing.

The shape is right: saving positive below k^* , negative above, and the **zero crossing at k^*** . The curve is smooth — no staircase, because there is no grid.

Compare 4.B5, whose grid search quantised saving into two values across 36 states. That artefact is simply gone. **Smoothness is a genuine and free gain here.**

Ledger fig_nn_saving, middle shock state. Deviation, not level, per the house rule. Code: [s07_generate.py](#).

Where the Error Actually Lives



Where	$\log_{10}$ err
at $0.5k^*$ (the edge)	-1.58
at k^*	-3.63
at $1.5k^*$ (the edge)	-1.96

Two decades — a factor of a hundred — separate the middle of the band from its edges, and the single worst point is at $k = 14.17$ — exactly the lower boundary. Drop

the outer 5% at each end and the maximum improves from -1.61 to -2.54.

Ledger worked `nnlgrowth`, `fig_nn_errk`. The lesson generalises: **a sampled method is weakest at the edge of the region it sampled, so *sample wider than you intend to evaluate***. Code: [s07_generate.py](#); L07 Part 5.

One Model, Five Methods

Method	numbers stored	seconds	max err, band	max err, near k^*
global VFI (4.B5)	2,100	0.941	-2.51	-2.82
perturbation, 1st order	4	0.000	-2.56	-4.01
perturbation, 2nd order	12	0.020	-4.50	-6.14
Chebyshev collocation	56	0.133	-6.40	-6.82
neural network (7.B1)	8,961	343.257	-1.61	-3.25

Chebyshev wins on every axis at once: 56 numbers, a tenth of a second, and five decades better than the network across the band. On this problem it is not close.

The first four rows are **read from the Session 5 ledger**, not recomputed, so the comparison cannot drift as either side is edited.

Ledger five_methods; rows 1-4 from s05_numbers.json key three_methods. Errors are $\log_{10}$ of the maximum unit-free Euler residual over 2,000 off-grid states.

Code: [s07_generate.py](#), [s05_generate.py](#).

Read That Table Before You Believe the Hype

The network came last, and it deserved to. It stored 8,961 numbers against Chebyshev's 56, took 2581 times as long, and was about **five decades** less accurate across the band.

This is not a failure of the implementation. **It is exactly what 7.A2 predicted:** on a smooth, low-dimensional function a spectral basis is close to optimal, and an adaptive basis has nothing to adapt to. The same table appeared there with $\sin$ and $e^{-x} \sin 3x$ in place of a policy function.

So why learn the method?

Because **every column of that table scales differently.** Chebyshev's 56 coefficients become 56^d — or a Smolyak set that still grows. The network's 8,961 parameters become 8,961 plus one row of weights per added state. **The crossover is not at $d = 1$.**

It is out where the classical methods stop running at all.

And the honest corollary: if a classical method can solve your model, use it. The network is for the models where none can.

Takeaways, and What 7.B2 Changes

- An economic model supplies its own training target: the **label on every sampled state is zero**, and the sample is unlimited.
- The loss $\int \mathcal{R}^2 D(k) dk$ is 5.B5's least-squares weighted residual, with $w = D$. **The sampling distribution is the weight function.**
- Feasibility belongs in the **parameterisation**, not in a clip: $c = \sigma(\cdot) \times \text{cash}$.
- Measured: the network is **last of five** on this model, and its error is worst **at the edge of the sampled band** — two decades worse than at k^* .

Bridge to 7.B2

This deck needed the **first-order conditions**. Many models do not have usable ones — kinks, discrete choices, occasionally binding constraints. **7.B2** goes back to the Bellman equation instead and trains *two* networks, a value and a policy, against each other. No FOCs required — and a new problem in their place.